FAST API

INTRODUCTION

- Fast API is a modern ,fast(high performance),web framework for building APIs with python based on standard Python type hints.
- Key features of using Fast API are:

1. Fast :Very high performance,on par with Nodejs and Go
2. Fst to code: Increases the speed to develop features by 200% to 300%.
3. Fewer bugs: Reduce about 40% of human induced errors.
4. Intuitive : Great editor support
5. Easy :Designed to be easy to use and learn .
6. Short: minimises code duplication.
7. Robust:Get production-ready code.
8. Standards-based:Based on the standards for APIs .

PYTHON INSTALLATION REFRESHER

- Fist step is to install the latest python version on your system .
- Then you can use any ide for using python [eg. visual studios,pycharm,jupyter notebook,etc]

Abou IDE'S :-

What is an IDE?

- IDEs are simply a source code editor. This means an IDE will help and assist in writing software!
- Many of them have terminals and other useful build automation tools embedded within.
- IDEs have debugger tools that allow you to dive deep within the code to find bugs and create solutions

Why do developers use IDEs?

- IDEs allow developers to begin work quickly and efficiently due to all the tools that come out of the box
- IDEs also parse all code, and helps find bugs before they are committed by human error
- The GUIs of IDEs is created with one goal in mind, help developers be efficient.

1. Variables:

- Variables are containers used for storing data values.
- Python has no commands for declaring a variable.
- A variable is created at the moment you first assign a value to it.

```
cost = 10
```

2. Comments:

- Comments can be used to explain the python code.
- Comments can be used to make the code more readable.
- Comments can be used prevent execution when testing the code.
- Python comments uses # symbol before the text.

```
# Going to print hello world!
print("Hello World")
```

3. String:

- String in python is a data type
- In python strings are surrounded by either single quotation marks or double quotation marks.

4. Getting user input:

- In python for getting user input we use input method .
- Input method only string data as input.
- By using various functions we can take different types of input.

5. Lists:

```
my_list = [80, 96, 72, 100, 8]
print(my_list)
```

- Lists are used for storing multiple items in a single variable.
- Lists are created by using square brackets “[]”.
- List items are ordered,changeable,and allows duplicate values.
- List items are indexed,the first item has index[0].
- Lists are ordered ,it means the order of items have a definite order and that order will not change.
- For inserting any element in list we can use insert method.
- For appending any element in the end of the list we use append method.
- Remove method is used for remove item of list at a particular location.
- Pop method is also used for removing the item from the list.
- Sort method is used for sorting the elements of the list.,etc.

6. Sets:

```
my_set = {1, 2, 3, 4, 5, 1, 2}
print(my_set)
```

- Sets are similar to list but are unordered and cannot contain duplicate items.
- Uses curly brackets for inserting elements.
- A set is collection which is unordered ,unchangeable and unindexed.

7. Tuple:

```
my_tuple = (1, 2, 3, 4, 5)
print(len(my_tuple))
```

- Tuples are used to store multiple items in single variable.
- Tuple is a collection which is ordered and unchangeable.
- Tuples are written with round brackets and allows duplicate values.

8. Booleans:

- Boolean represents one of two values: True and False
- Whenever we compare any values then we get answer as true or false.

9. Operators:

- Operators are used for performing operations on variables and values.
- In python we have following types of operators :

  - Arithmetic operators
  - Assignment operators
  - Comparison operators
  - Logical operators
  - Identity operators
  - Membership operators
  - Bitwise operators

10. If.....Else

- If else is an conditional statement that is used for checking conditions.
- Along with if else we sometimes uses elif when you have one than two conditions for checking.

```
hour = 21

if hour < 15:
    print("Good morning!")
elif hour < 20:
    print("Good afternoon!")
else:
    print("Good Night!")
```

11. For loop:

- A for loop is used for iterating over a sequence(that is either a list, set or tuple, string).
- For loop are generally used as for in loop to directly to loop through elements .
- We can loop through range of elements by using range function.

```
my_list = ["Monday", "Tuesday", "Wednesday", "Thursday"]
for x in my_list:
    print(f"Happy {x}!")
```

12. While loop:

- While loop is a looping statement that continues looping until a condition is met .
- While is written with a condition and until that condition is met the loop continues iterating but when the condition becomes false you break out while loop

```
i = 0

while i < 5:
    i += 1
    if i == 3:
        continue
    print(i)
    if i == 4:
        break
```

13. Dictionary:

- Dictionary are used to store data values in key:value pairs.
- A dictionary is a collection which is ordered ,changeable and do not allow duplicates.

```
user_dictionary = {
    'username': 'codingwithroby',
    'name': 'Eric',
    'age': 32
}


for x, y in user_dictionary.items():
    print(x, y)
```

14. Function:

- A function is block of code that performs certain particular task .
- A function only executes only when it is called.
- We can pass data, known as parameters into a function.
- For creating a function python we use **def** keyword.

```python
def my_function():
    print("Inside my_function")

my_function()
```

15. Import :

- Import command is used for importing modules or specific function,class or variables to be used in your file.
- There are some standard module can be used within python page by importing them.

```python
def calculate_homework(homework_assignments_arg):
    sum_of_grades = 0
    for homework in homework_assignments_arg.values():
        sum_of_grades += homework
    final_grade = round(sum_of_grades /
                        len(homework_assignments_arg),
    print(final_grade)
```

The file that we are importing program

```python
import Imports.grade_average_service as grade_se

homework_assignment_grades = {
    'homework_1': 85,
    'homework_2': 100,
    'homework_3': 81
}


grade_average_service.calculate_homework(
    homework_assignment_grades
)
```

OBJECT ORIENTED PROGRAMMING

- Object oriented programming is a programming paradigm based on the concepts of objects,which can data and code.
- Objects are data that is defined by the properties and methods.
- There are four pairs of OOP.
- encapsulation,abstraction,inheritance,polymorphism.

1. ABSTRACTION:

- This means to hide the implementation and only show necessary details to the user.

- This allows users to not have to understand what the functionality is behind scenes.
- You can create simple and reusable code
- Allows for a better use of the DRY principle (Don't repeat yourself)
- Enables Python objects to become more scalable.

CONSTRUCTOR:

- Are used to create and intialize an object of a class with or without starting value.
- Types of constructor:

1. Empty constructor
2. No argument constructor
3. Argument constructor

2. Encapsulation:

- Bundling the data.
- Thataking the class public or private .
- And this is done by using double underscore around init function constructor.
- We can also use get and set function.
-

File: Enemy.py

GETTERS & SETTERS

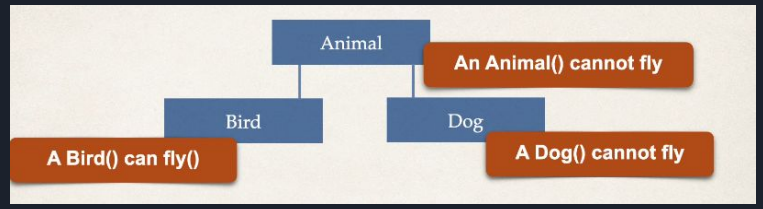
```
class Enemy:

    def __init__(self, type_of_enemy, health_points=10, attack_damage=1):
        self.__type_of_enemy = type_of_enemy
        self.health_points = health_points
        self.attack_damage = attack_damage

    def get_type_of_enemy(self):
        return self.__type_of_enemy

    def set_type_of_enemy(self, type_of_enemy):
        self__type_of_enemy = type_of_enemy

    def talk(self):
        print('I am an Enemy!')
```

Why use Encapsulation?

- Helps keep related fields and methods together
- Makes our code cleaner and easier to read
- Provides more flexibility to our code
- Provides more reusability with our code

3. INHERITANCE:

- Process of acquiring properties or function from one class to other classes
- It is used for creating hierarchy between classes.
- In inheritance method overriding is also possible ,it allows us to create a our own methods with same name in the child instead of using the parent method.



#note:
Self vs super:

- Self is used to refer to the current object that is created or being instantiated ,while super is used to refer to the parent class.
- Self is used when there is a need to differentiate between the instance variable and parameters with the same name,while super is used to call the parent class methods and/or constructors.

4. POLYMORPHISM:

- Polymorphism is a way represent class in different ways.

FASTAPI SETUP AND INSTALLATION

Python virtual environments:

- A virtual environment is a python environment that is isolated from those in other python environments.

How to install Dependencies:

- Pip is a python package manager.
- Pip is used for install and update packages.

```
ericroby@Erics-MacBook-Pro ~% python3 -m pip --version
pip 22.3.1
```

Setting up virtual environment:

- I started by creating a brand-new folder / directory called "FastAPI"
- Within our pip we will be checking what dependencies we already have installed.
- Python already has an included dependency for virtual environments called venv.
- Creating a new FastAPI environment as a virtual environment
- Activating our virtual environment to install our dependencies in

Fast api installation on windows:

- Step 1: open cmd on your system
- Step 2: make directory in your **C:\Users\somename\Documents>** file and name it as fastapi.

```
C:\Users\Ericr\Documents>mkdir fastapi

C:\Users\Ericr\Documents>cd fastapi
```

- Step 3: Creating fastapi environment.

```
C:\Users\Ericr\Documents\fastapi>python -m venv fastapienv
```

- Step 4:then activate fastapienv to install fastapi.

```
C:\Users\Ericr\Documents\fastapi>fastapienv\Scripts\activate.bat

(fastapienv) C:\Users\Ericr\Documents\fastapi>_
```

- Step 5: to install fastapi and uvicorn [uvicorn is webserver for fastapi]

```
pip install fastapi
```

```
>pip install "uvicorn[standard]"
```

- Step 6: If you want to deactivate fastapi env than use deactivate command.

```
(fastapienv) C:\Users\Ericr\Documents\fastapi>deactivate
C:\Users\Ericr\Documents\fastapi>
```

PROJECT 1-BOOKS

- **Starting with Get:**
- Get is an http request method.
- For using fast api first step is to import fastapi
- Then creating an app instance to using fastapi within application.
- By using the same instance you can create get method .
- .get decorator declare that the read_root function will handle get requests a root path or you can also specify the path .
- Then you will have a asyn function within the get method that is executed when the get request hits the path. It returns a dictionary ,which fastapi automatically converts to jason in the response.

```
from fastapi import FastAPI
app = FastAPI()
```

```
@app.get("/books")
async def read_all_books():
    return BOOKS
```

- Then we can enhance the normal template code for our project.
- So we will create a list of books with title ,author and category of the books.
- We can use swagger by writing the url with /docs

127.0.0.1:8000/docs

- For executing the file we will the command mentioned below:

```
% uvicorn books:app --reload
```

- **PATH PARAMETERS:**
- Path parameters are requested parameters that have been attached to the url.
- Path parameters are usually defined as a way to find information based on location.

```
/Users/codingwithroby/Documents/python/fastapi/section1
```

- In the example of path given above we are expecting to see section1 in this path.
- Dynamic_param is used for handling dynamic request.the dynamic_param is used for matching the path parameters.

```python
@app.get("/books/{dynamic_param}")
async def read_all_books(dynamic_param):
    return {'dynamic_param': dynamic_param}
```

dynamic_param * required
(path)
science

```
http://127.0.0.1:8000/books/science
```

- We can also explicitly give the data type as well.
- Order matter in rendering the request if you are calling dynamic parameter above static parameter no matter what you dynamic parameter is going to be working
- For executing the static and dynamic parameter you need to static before dynamic parameter function.
- By using the dynamic parameters and for loop we can return the specific book.[%20=space in url]

```python
@app.get("/books/{book_title}")
async def read_book(book_title: str):
    for book in BOOKS:
        if book.get('title').casefold() == book_title.casefold():
            return book
```

- **QUERY PARAMETER:**
- Query parameters are request parameters that are being attached after a “?”.
- Query parameters have name=value pair relationship.
- In the url whatever you will enter that is going the parameter you will pass in the async function.
- In our example we want to filter the books by category.

```
@app.get("/books/")
async def read_category_by_query(category: str):
    books_to_return = []
    for book in BOOKS:
        if book.get('category').casefold() == category.casefold():
            books_to_return.append(book)
    return books_to_return
```

- We can also use path and query parameter together.

- **POST:**
- Used to create data.
- Post can have a body that has additional information that get does not have so that fastapi application can use it.

```
@app.post("/books/create_book")
async def create_book(new_book=Body()):
    BOOKS.append(new_book)
```

- **PUT method:**
- Used to update data.
- Put can have a body that as additional information(like POST)that get does not have.

```
@app.put("/books/update_book")
async def update_book(updated_book=Body()):
    for i in range(len(BOOKS)):
        if BOOKS[i].get('title').casefold() == updated_book.get('title').casefold():
            BOOKS[i] = updated_book
```

- **DELETE METHOD**
- Used for delete data.

```
@app.delete("/books/delete_book/{book_title}")
async def delete_book(book_title: str):
    for i in range(len(BOOKS)):
        if BOOKS[i].get('title').casefold() == book_title.casefold():
            BOOKS.pop(i)
            break
```

PROJECT 2 -BOOKS

- In project 2 we will add new information .
- And we add: Data validation,Exception handling,status codes,Swagger configuration,Python request objects.

- Pydantic:
- Pydantic is a python library used for data validation,serialisation and setting management ,primary leveraging python type hints
- It allows developers to define data structures using python classes and automatically validate incoming dara against these structures
- FastAPI is compatible with both pydatic v1 and pydantic v2 .
- Pydantic is commonly used as a resource for data validation and how to handle data.

We will be implementing Pydantics

- Create a different request model for data validation
- Field data validation on each variable / element

```
class BookRequest(BaseModel):
    id: int
    title: str = Field(min_length=3)
    author: str = Field(min_length=1)
    description: str = Field(min_length=1, max_length=100)
    rating: int = Field(gt=0, lt=5)
```

BookRequest and Book Conversion

- We will convert the Pydantics Request into a Book
- Here is an example within an upcoming Post Request Method

```
@app.post("/create-book")
async def create_book(book_request: BookRequest):
    new_book = Book(**book_request.dict())
    BOOKS.append(new_book)
```

** operator will pass the key/value from BookRequest() into the Book() constructor

- We can create a new post method for taking data by using body() method but the body method do not allows us to do any validation .

```
@app.post("/create-book")
async def create_book(book_request=Body()):
    BOOKS.append(book_request)
```

- We will use pydantic and we get pydantic pre installed with fastapi
- We are going to Base Model model of pydatic for validating the variable within the object.
- So for validation we will create a new book_request object to validate the new request coming in,so if the book request matches with the validation then it is transformed into book and be saved in the book list.

```
@app.post("/create-book")
async def create_book(book_request: BookRequest):
    BOOKS.append(book_request)
```

```
class BookRequest(BaseModel):
    id: int
    title: str
    author: str
    description: str
    rating: int
```

- Instead of using body method we can use the class book_request is a pydantic basemodel class.The in the above snippet book_request an book request type of object but we want it to be of book.
- Then for adding data validation we will use Field method from pydatic

```
class BookRequest(BaseModel):
    id: int
    title: str = Field(min_length=3)
    author: str = Field(min_length=1)
    description: str = Field(min_length=1, max_length=100)
    rating: int = Field(gt=0, lt=6)
```

- For making id to be unique and the value of id to be incrementing automatically every time a new book is added for that we can create a function.

```python
@app.post("/create-book")
async def create_book(book_request: BookRequest):
    new_book = Book(**book_request.dict())
    BOOKS.append(find_book_id(new_book))


def find_book_id(book: Book):
    book.id = 1 if len(BOOKS) == 0 else BOOKS[-1].id + 1
    return book
```

- We can also make our id as optional field by using optional class of typing module
- We can also set the field value for description and setting up the default value

```python
class BookRequest(BaseModel):
    id: Optional[int] = None
```

```python
id: Optional[int] = Field(description='ID is not needed on create', default=None)
```

- We can pre-populate the value by using model_config inside pydantic book_request

```python
model_config = {
    "json_schema_extra": {
        "example": {
            "title": "A new book",
            "author": "codingwithroby",
            "description": "A new description of a book",
            "rating": 5
        }
    }
}
```

- Fetching single book by giving input of id

```python
@app.get("/books/{book_id}")
async def read_book(book_id: int):
    for book in BOOKS:
        if book.id == book_id:
            return book
```

- Fetch books on the basis of rating by using query parameter.

```python
@app.get("/books/")
async def read_book_by_rating(book_rating: int):
    books_to_return = []
    for book in BOOKS:
        if book.rating == book_rating:
            books_to_return.append(book)
    return books_to_return
```

- Updating Book by using put request method on the basis of the id.

```python
@app.put("/books/update_book")
async def update_book(book: BookRequest):
    for i in range(len(BOOKS)):
        if BOOKS[i].id == book.id:
            BOOKS[i] = book
```

- Deleting a book by using delete request method and on the basis of id

```python
@app.delete("/books/{book_id}")
async def delete_book(book_id: int):
    for i in range(len(BOOKS)):
        if BOOKS[i].id == book_id:
            BOOKS.pop(i)
            break
```

- Adding data validation for the path parameters ,we can do it by using Path class by importing it from fastapi

```python
@app.get("/books/{book_id}")
async def read_book(book_id: int = Path(gt=0)):
    for book in BOOKS:
```

- Adding validation to a query parameter,it is to be done by using Query method again imported from fastapi

```python
@app.get("/books/")
async def read_book_by_rating(book_rating: int = Query(gt=0, lt=6)):
    books_to_return = []
```

- Status code :
- An HTTP status code is used to help the client to understand what happened on the server application.
- Status code are the international standards on how a client/server should handle the result of a request.
- It allows everyone who sends the request to know if their submission was successful or not.

Status Codes:

1xx	Information Response: Request Processing.
2xx	Success: Request Successfully complete
3xx	Redirection: Further action must be complete
4xx	Client Errors: An error was caused by the client.
5xx	Server Errors: An error occurred on the server.

2xx Successful Status Codes:

200: OK	Standard Response for a Successful Request. Commonly used for successful Get requests when data is being returned.
201: Created	The request has been successful, creating a new resource. Used when a POST creates an entity.
204: No Content	The request has been successful, did not create an entity nor return anything. Commonly used with PUT requests.

-

400: Bad Request	Cannot process request due to client error. Commonly used for invalid request methods.
401: Unauthorized	Client does not have valid authentication for target resource
404: Not Found	The clients requested resource can not be found
422: Unprocessable Entity	Semantic Errors in Client Request

5xx Server Status Codes:

500: Internal Server Error	Generic Error Message, when an unexpected issue on the server happened.

- **HTTP EXCEPTION:**
- HTTP exception is something that we have to raise within a method,which cancel the functionality of the method and return a message in a status code back to our user
- To use the http exception we will import from fastapi library.

```
from fastapi import FastAPI, Path, Query, HTTPException
```

- In the following example are raising the exception that return status code 404 with a message.

```
@app.get("/books/{book_id}")
async def read_book(book_id: int = Path(gt=0)):
    for book in BOOKS:
        if book.id == book_id:
            return book
    raise HTTPException(status_code=404, detail='Item not found')
```

- We can also create an http exception for handling wrong input

```
async def update_book(book: BookRequest):
    book_changed = False
    for i in range(len(BOOKS)):
        if BOOKS[i].id == book.id:
            BOOKS[i] = book
            book_changed = True
    if not book_changed:
        raise HTTPException(status_code=404, detail='Item not found')
```

- Explicit status code response for successful API endpoints
- We can give different successful response instead of only giving 200 status code as successful code we can define different status code.
- We can do it by using status method by importing it from stralette.

```
@app.get("/books", status_code=status.HTTP_200_OK)
async def read_all_books():
```

```
@app.post("/create-book", status_code=status.HTTP_201_CREATED)
async def create_book(book_request: BookRequest):
```

PROJECT 3-TODOS

- In this project we will do database for storing data.
- And we will also authentication and authorisation of the users.

- Project three we will be switching our focus to TODOS instead of BOOKS
- New Information will include:
  - Full SQL Database
  - Authentication
  - Authorization
  - Hashing Passwords

Creating a Todo Table

- We will create new Todo Table Models for our application
- We will be using these Todos to save records throughout this project

- **SQL database:**

- **What is a databases?**

- Organised collection of structured information of data which is stored in a computer system
- The data can be easily accessed.
- The data an be modified
- The data can be controlled and organized.
- Many databases uses a structured query language to modify and write data.

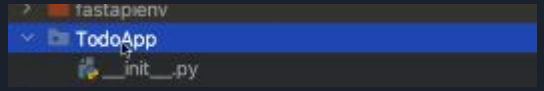
- What is data in database?

- Data can be related to just about any object.
- Since data is an independent entity
- Database are organised in how the data can be retrieved ,stored and modified .
- For maintaining database we use a software that is called as DBMS (database management system)
- Some of the very popular dbms systems are Oracle,postgresql.mysql,etc.
- Whereas SQL standards for structured query language.
- SQL s used for doing different things and mainly the CRUD operation.

Creation of TODOSAPP project:

- Todos is a project where we will the list of item that we want to do with a feature to check and checkoff the task.
- And we are also going to have the ability to prioritise the docs.

Project creation



- Step one is to create a todos python package.
- Step 2 is to create a new python file with a name database.py
- This database .py is to used in order to be able to create URL string which will connect fastapi application with database.
- Change the direct to the package that we created where we will create the file
- Then the next step is to install sqlalchemy .
- Sqlalchemy is an ORM(Object relational mapping) , that is what fastapi application is going to use to be able to create a database and is used for creating the connection with database.

```
odoApp % pip install sqlalchemy
```

- Creating Database.py file in TodosApp:
- First step is to create a variable and this variable is going to have the value of the location of the database on the fastapi application.

```
SQLALCHEMY_DATABASE_URL = 'sqlite:///./todos.db'
```

- Next step is to create a database engine for the application that we will use for opening the connection and be able to use the database

```
m sqlalchemy import create_engine

ALCHEMY_DATABASE_URL = 'sqlite:///./todos.db'

ine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={'check_same_thread': False})
```

- Next step is to create an instance session local that is going to be the database.
- For that we will import session_maker from sqlalchemy.orm and he we will have to bind it with the engine that we created.

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

- Then we have to create a declarative base that will create a base s for declarative model in SQLalchemy.
- This base class acts as an central point for defining database table and their corresponding python classes.

```
from sqlalchemy.ext.declarative import declarative_base
```

```
Base = declarative_base()
```

- Next step is to create a model.py file inside your TodosApp folder.

- Creating Main.py file:
- This main.py file is used for definition the type of database we need to create.
- The first step is to import the base we created in the database.py file into your models.py so that it define that we are creating the model for database.py file.
- Then we will create a class and name it as todos where we will define the name of the table
- This class todos is going to mimic the table of todos,the you can create the columns of the table.
- This is the todos table .

Id (PK)	title	description	priority	complete
1	Go to store	To pick up eggs	4	0
2	Haircut	Need to get length 1mm	3	0
3	Feed dog	Make sure to use new food brand	5	0
4	Water plant	Inside and Outside plants	4	0
5	Learn something new	Learn to program	5	0

- In the table we will create the columns we want by importing the datatypes from sqlalchemy

```python
from sqlalchemy import Column, Integer, String, Boolean
```

```python
class Todos(Base):
    __tablename__ = 'todos'

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String)
    description = Column(String)
    priority = Column(Integer)
    complete = Column(Boolean, default=False)
```

- Creating Todos database:
- Start by creating a main.py file that is going to be the root file where we will create our fastapi application.ui

```python
from fastapi import FastAPI
import models
from database import engine

app = FastAPI()

models.Base.metadata.create_all(bind=engine)
```

- Installing SQLite3 :
- We can install sqlite3 so that we can manipulate the data of a database,and will be able to use database from terminal.

- Basic SQL queries:
- Inserting data into table(todos):

```
INSERT INTO todos (title, description,
priority, complete)

VALUES ('Go to store', 'To pick up
eggs' 4, False);
```

- To display the database we use SELECT query:
  In this image we are displaying only titles of the data from table.

```
SELECT title FROM todos;
```

- Where clause: it is used for filtering the data and used for displaying the rows by defining the conditions
  Here we are displaying all the columns and rows where id=2

```
SELECT * FROM todos WHERE title='Feed dog';
```

Select ALL rows & columns WHERE title= Feed dog

- Update clause: this clause is used for updating the data values in your table.

```
UPDATE todos SET complete=True WHERE id=5;
```

Here we are specifying to update all the rows and columns of table todos and set the value of complete as to be true where id =5.

- DELETE clause:this is used for delete a row data from your table.

Here we are deleting the row where id=5

```
DELETE FROM todos WHERE id=5;
```

- GET method for fetching the records of todos.db file:
- Creating db dependency : by creating a function we can create a database dependency by using yield .
- Yield means that the code prior to yield statement including wild statement is executed before sending the response.
- And the code following the yield statement is executed after the response.
- This makes fastapi faster as we can fetch information from the database and then return it to the client and can close the connection afterwards.

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

- Creating an async api endpoint:
- We will create the endpoint by using a dependency injection.
- *Note :Dependency injection is a software where classes or functions receives its dependencies from another resource.

```
@app.get("/")
async def read_all(db: Annotated[Session, Depends(get_db)]):
    return db.query(Todos).all()
```

- Get method for fetching single data value:
- We will do it by using filter method for filtering the id match.

```
@app.get("/todo/{todo_id}", status_code=status.HTTP_200_OK)
async def read_todo(db: db_dependency, todo_id: int = Path(gt=0)):
    todo_model = db.query(Todos).filter(Todos.id == todo_id).first()
    if todo_model is not None:
        return todo_model
    raise HTTPException(status_code=404, detail='Todo not found.')
```

- POST method for creating and saving a new todo:
- This is to be done by using pydantic base Model
- For that first step is to create a todo_request class .

```
@app.post("/todo", status_code=status.HTTP_201_CREATED)
async def create_todo(db: db_dependency, todo_request: TodoRequest):
    todo_model = Todos(**todo_request.dict())

    db.add(todo_model)
    db.commit()
```

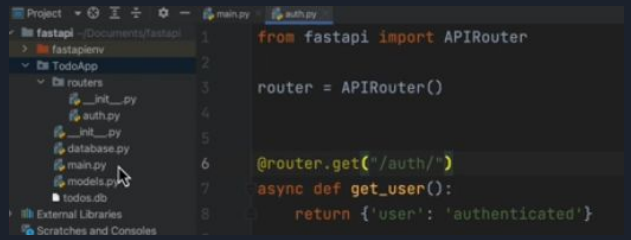
- PUT method for updating the todo list:
- This method is used for updating the database element on the basis of the id

```
models.py
@app.put("/todo/{todo_id}", status_code=status.HTTP_204_NO_CONTENT)
async def update_todo(db: db_dependency,
                      todo_id: int,
                      todo_request: TodoRequest):
    todo_model = db.query(Todos).filter(Todos.id == todo_id).first()
    if todo_model is None:
        raise HTTPException(status_code=404, detail='Todo not found.')

    todo_model.title = todo_request.title
    todo_model.description = todo_request.description
    todo_model.priority = todo_request.priority
    todo_model.complete = todo_request.complete

    db.add(todo_model)
    db.commit()
```

- DELETE method for deleting a task from database

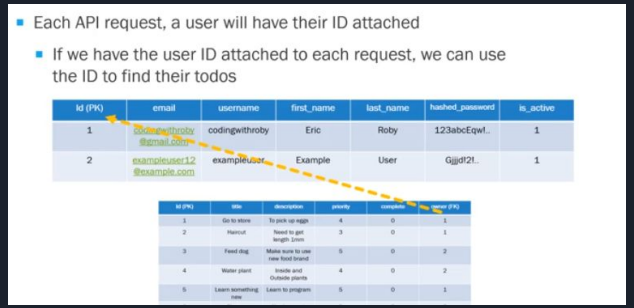
```
@app.delete("/todo/{todo_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_todo(db: db_dependency, todo_id: int = Path(gt=0)):
    todo_model = db.query(Todos).filter(Todos.id == todo_id).first()
    if todo_model is None:
        raise HTTPException(status_code=404, detail='Todo not found.')
    db.query(Todos).filter(Todos.id == todo_id).delete()

    db.commit()
```

- Authentication and Authorisation:
- For authentication we will be creating a different py file ,and this file will run on a different port than that of main.py file.
- But to scale the authentication.py file and main.py file together we will use something called router,so that we can run both the files on the same port .
- For scaling we will firstly create a new router package,inside which you will create a auth.py file.



- Then we will create another py file in the router package that will include all the api endpoints instead of main.py
- So that main.py only defines the database and includes the router files.
- This step helps us to clean our application for maintainability and scalability of the application.

- **Foreign key in database:**
- Foreign key is a column within a relational database table that provides a link between two separate tables.
- A foreign key references a primary key of another table.
- Most of the relational databases uses foreign key for linking table together to present data.
- In our project we will creating a new user table that is to be linked to the todos table.



- **Creating user table:**
- Sqlalchemy cannot enhance the table ,for enhancing the database we need to alembic .
- SQlalchemy is only used for creating a database and table
- Within your models.py file you will have to change the classes and have to create user table.

```
class Users(Base):
    __tablename__ = 'users'

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True)
    username = Column(String, unique=True)
    first_name = Column(String)
    last_name = Column(String)
    hashed_password = Column(String)
    is_active = Column(Boolean, default=True)
    role = Column(String)
```

- Creating users for user table:
- We will start by creating a pydantic class

```python
class CreateUserRequest(BaseModel):
    username: str
    email: str
    first_name: str
    last_name: str
    password: str
    role: str
```

- For creating a new user data we will use post method.

```python
@router.post("/auth")
async def create_user(create_user_request: CreateUserRe
    create_user_model = Users(
        email=create_user_request.email,
        username=create_user_request.username,
        first_name=create_user_request.first_name,
        last_name=create_user_request.last_name,
        role=create_user_request.role,
        hashed_password=create_user_request.password,
        is_active=True
    )
```

- Hashing password
- We will use bcrypt it is very popular way to hash text.
- And then we will store the hashed password in the database
- For this the first is to install passlib and bcrypt through your terminal.
- These two will work together

```
pip install bcrypt==4.0.1
) % pip install passlib
```

```python
from passlib.context import CryptContext

router = APIRouter()

bcrypt_context = CryptContext(schemes=['bcrypt'], deprecated='auto')
```

- Passlib is a comprehensive password hashing library for Python, designed to handle various password hashing schemes and provide a secure and flexible way to manage user passwords.
- The `CryptContext` class is a central component of Passlib. It provides a flexible and configurable interface for managing password hashing, allowing developers to define and use multiple hashing schemes, handle password upgrades, and perform other related operations.

- And for hashing the password by using bcrypt context we will hash the user entered password

```
role=create_user_request.role,
hashed_password=bcrypt_context.hash(create_user_request.password),
```

- Saving user data in database:
- For saving the user request in the table we will use the same method that is by creating the db session and yielding it.

```
@router.post("/auth", status_code=status.HTTP_201_CREATED)
async def create_user(db: db_dependency,
                      create_user_request: CreateUserRequest):
    create_user_model = Users(
        email=create_user_request.email,
        username=create_user_request.username,
        first_name=create_user_request.first_name,
        last_name=create_user_request.last_name,
        role=create_user_request.role,
        hashed_password=bcrypt_context.hash(create_
        is_active=True
    )

    db.add(create_user_model)
    db.commit()
```

- Authenticating a valid user:
- For this we will create another post request method api endpoint that is going to have a path of /token.
- Token is what we are going to return back to the user that is going to have all the information about the user inside
- Is token is going o be JWT that is json web token
- For user authentication we will be using OAuth form for taking username and password.
- For using it first step is to install fastapi multipart

```
% pip install python-multipart
```

- Then we will import it

```
from fastapi.security import OAuth2PasswordRequestForm
```

- For authentication we will a function that we will pass in the apienpoint for checking authentication.

```python
def authenticate_user(username: str, password: str, db):
    user = db.query(Users).filter(Users.username == username).first()
    if not user:
        return False
    if not bcrypt_context.verify(password, user.hashed_password):
        return False
    return True
```

```python
@router.post("/token")
async def login_for_access_token(form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
                                 db: db_dependency):
    user = authenticate_user(form_data.username, form_data.password, db)
    if not user:
        return 'Failed Authentication'
    return 'Successful Authentication'
```

- JWT(JSON WEB TOKEN):
- Json web token is a self-contained way to securely transmit data and information between two parties(between user and the admin) using json objects.
- JSON web token can be trusted as each JWT can be digitally signed ,which in return allows the server to know if the JWT has been changed at all.
- JWT should be used when dealing with authorization.
- JWT is a great for information to be exchanged between the server and the client.
- JWT is one of the most popular bearer token .
- When a user logs in a server will return a Jwt which is a string of characters when decoded shows information about client and user.
- Json web token structure :

A JSON Web Token is created of three separate parts separated by dots (.) which include:

- Header : (a)
- Payload : (b)
- Signature : (c)

```
aaaaaaaa.bbbbbbbb.cccccccc
```

- The first part that jwt header:

- A JWT header usually consist of two parts:
  - (alg) The algorithm for signing
  - "typ" The specific type of token
- The JWT header is then encoded using Base64 to create the first part of the JWT (a)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

```
aaaaaaaa.bbbbbbbb.cccccccc
```

- The second part that is jwt payload :
- It also comprises of expiration exp
- Exp is optional field but is very useful

- A JWT Payload consists of the data. The Payloads data contains claims, and there are three different types of claims.
  - Registered
  - Public
  - Private
- The JWT Payload is then encoded using Base64 to create the second part of the JWT (b)

```
{
  "sub": "1234567890",
  "name": "Eric Roby",
  "given_name": "Eric",
  "family_name": "Roby",
  "email": "codingwithroby@gmail.com"
  "admin": true
}
```

```
aaaaaaaa.bbbbbbbb.cccccccc
```

- The third that is jwt signature:

- A JWT Signature is created by using the algorithm in the header to hash out the encoded header, encoded payload with a secret.
- The secret can be anything, but is saved somewhere on the server that the client does not have access to
- The signature is the third and final part of a JWT (c)

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret)
```

```
.........
```

- Encoding JWT:
- Instead of returning messages we will be returning Jwt token for successful user authentication or failed authentication.
- First we need to install a library

```
% pip install "python-jose[cryptography]"
```

- Then we will include some functionality

```
from jose import jwt
```

- Then the next step to create a secret key this secret key is generally have to be a super secretive key.
- For that we can use

```
App % openssl rand -hex 32
```

this is going to give a random 32 bit hex string.

- Then for algorithm we will HS256 algorithm that is a common algorithm or hashing data ensuring data integrity and authenticity.
- Then we will create another function .
- For this we will use

```
code = {'sub': username, 'id': user_id}
pires = datetime.utcnow()
```

datetime.now(timezone.utc)

- Function:

```
def create_access_token(username: str, user_id: int, expires_delta: timedelta):
    encode = {'sub': username, 'id': user_id}
    expires = datetime.utcnow() + expires_delta
    encode.update({'exp': expires})
    return jwt.encode(encode, SECRET_KEY, algorithm=ALGORITHM)
```

- In the api endpoint we will make some changes :

```
@router.post("/token", response_model=Token)
async def login_for_access_token(form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
                                 db: db_dependency):
    user = authenticate_user(form_data.username, form_data.password, db)
    if not user:
        return 'Failed Authentication'
    token = create_access_token(user.username, user.id, timedelta(minutes=20))

    return {'access_token': token, 'token_type': 'bearer'}
```

- Decoding JWT:
- Decoding of jwt is used for verifying whether a jwt is authenticated or not.
- For this we will import OAuth2Passwordbearer from fastapi.security

```
oauth2_bearer = OAuth2PasswordBearer(tokenurl='token')
```

- We will create a new function for authentication.

```
async def get_current_user(token: Annotated[str, Depends(oauth2_bearer)]):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get('sub')
        user_id: int = payload.get('id')
        if username is None or user_id is None:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                                detail='Could not validate user.')
        return {'username': username, 'id': user_id}
    except JWTError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                            detail='Could not validate user.')
```

- DIVIDING AUTH AND TODOS API DIFFERENTLY

```
router = APIRouter(
    prefix='/auth',
    tags=['auth']
)
```

- POST todo with user authentication:
- In the todos file we will include auth get_current user an we will add it as dependency.
- This is doe for checking if the user is valid.
- And we will add user_id as the new owner id within your todos table.

```
@router.post("/todo", status_code=status.HTTP_201_CREATED)
async def create_todo(user: user_dependency, db: db_dependency,
                      todo_request: TodoRequest):
    if user is None:
        raise HTTPException(status_code=401, detail='Authentication Failed')
    todo_model = Todos(**todo_request.dict(), owner_id=user.get('id'))

    db.add(todo_model)
    db.commit()
```

- Get method with user authentication:

```
@router.get("/", status_code=status.HTTP_200_OK)
async def read_all(user: user_dependency, db: db_dependency):
    return db.query(Todos).filter(Todos.owner_id == user.get('id')).all()
```

- Get method to get the todo by id with user authentication:

```
@router.get("/todo/{todo_id}", status_code=status.HTTP_200_OK)
async def read_todo(user: user_dependency, db: db_dependency, todo_id: int = Path(gt=0)):
    if user is None:
        raise HTTPException(status_code=401, detail='Authentication Failed')

    todo_model = db.query(Todos).filter(Todos.id == todo_id)\
        .filter(Todos.owner_id == user.get('id')).first()
    if todo_model is not None:
        return todo_model
    raise HTTPException(status_code=404, detail='Todo not found.')
```

- PUT method with user authentication:

```python
async def update_todo(user: user_dependency, db: db_dependency,
                      todo_request: TodoRequest,
                      todo_id: int = Path(gt=0)):
    if user is None:
        raise HTTPException(status_code=401, detail='Authentication Failed')

    todo_model = db.query(Todos).filter(Todos.id == todo_id)\
        .filter(Todos.owner_id == user.get('id')).first()
    if todo_model is None:
        raise HTTPException(status_code=404, detail='Todo not found.')

    todo_model.title = todo_request.title
    todo_model.description = todo_request.description
    todo_model.priority = todo_request.priority
    todo_model.complete = todo_request.complete

    db.add(todo_model)
    db.commit()
```

- DELETE method with user authentication:

```python
@router.delete("/todo/{todo_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_todo(user: user_dependency, db: db_dependency, todo_id: int = Path(gt=0)):
    if user is None:
        raise HTTPException(status_code=401, detail='Authentication Failed')

    todo_model = db.query(Todos).filter(Todos.id == todo_id)\
        .filter(Todos.owner_id == user.get('id')).first()
    if todo_model is None:
        raise HTTPException(status_code=404, detail='Todo not found.')
    db.query(Todos).filter(Todos.id == todo_id).filter(Todos.owner_id == user.get('id')).delete()

    db.commit()
```

- Admin Route for Admin role:
- If the role of any of the user is going to be admin that user can access all the todos of other users as well.
- Within user table there was column role that takes the role of the user.
- This is to done by creating a new py file in router package and name it as admin with functionality.

```
        db.close()


db_dependency = Annotated[Session, Depends(get_db)]
user_dependency = Annotated[dict, Depends(get_current_user)]


@router.get("/todo", status_code=status.HTTP_200_OK)
async def read_all(user: user_dependency, db: db_dependency):
    if user is None or user.get('user_role') != 'admin':
        raise HTTPException(status_code=401, detail='Authentication Failed')
    return db.query(Todos).all()
```

- Production database:
- A production database is the live, real-world database used by an application or system to store and manage actual data being used in daily operations.

- Difference between production dbms and sqlite3:

- SQLite3 strives to provide local data storage for individual applications and devices.
- SQLite3 emphasizes economy, efficiency and simplicity.
- For most small / medium applications, SQLite3 will works perfectly.
- SQLite3 focuses on different concepts than a production Database Management System.

- MySQL & PostgreSQL focuses on a big difference compared to SQLite3.
- These production DBMS focuses on scalability, concurrency and control.
- If you application is going to have 10s of thousands of users, it may be wise to switch to a production DBMS
- If you application is only you, and a few others, SQLite3 will work great!

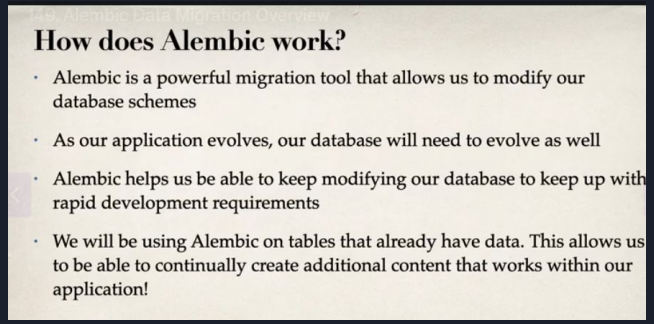
- Key notes(Production DBMS):
- Sqlite3 run in-memory or local disk ,which allows development of a sqlite3 data to be easy.as it is a part of your application.
- Production dbms runs on its own server and port.which means you need to make sure that the database is running and authenticated before using it
- For deployment you will deploy sqlite3 application along with the application.
- For deployment of production dbms you will need to deploy the database separately .

- POSTGRESQL:
- Production ready
- Open-source relational dbms
- It is secure
- Requires a server
- Scalable
- Steps to install postgresql in windows:
- Visit www.postgresql and download the postgresql installer suitable for window and the latest one.
- Setup you postgresql in your system and install it on the system
- Then start with pgadmin

- MYSQL:[BAL
- Open source relational dbms
- Requires a server
- Secure and scalable
- Install mysql
- Setup mysql
- Creating users and todos within todos database

PROJECT 3.5-TODOS WITH ALEMBIC

- Alembic:
- Lightweight migration for when using SQLAlchemy
- Migration tools allows us to plan ,transfer and upgrade resources within databases.
- Alembic allows you to change a sqlalchemy database table after it is been created.
- Currently sqlalchemy has a table but alembic will allows us to changes in the database table without deleting the table.



- To use alembic we will install alembic within the project.

```
croby@FastAPI-Project ~%     pip install alembic
```

- After installing alembic we will have to use some commands within our application

Alembic Command	Details
alembic init <folder name>	Initializes a new, generic environment
alembic revision -m <message>	Creates a new revision of the environment
alembic upgrade <revision #>	Run our upgrade migration to our database
alembic downgrade -1	Run our downgrade migration to our database

- After we initialize the project with alembic ,two new items will appear in the directory
- Alembic.ini
- Alembic directory
- These are created automatically by alembic so we can upgrade,downgrade and keep data integrity of our application

Alembic.ini file

- File that alembic looks for when invoked
- Contains a bunch of configuration information for Alembic that we are able to change to match our project

Alembic Directory

- Has all environmental properties for alembic
- Holds all revisions of your application
- Where you can call the migrations for upgrading
- Where you can call the migrations for downgrading

- First step is to install alembic.
- Then we will create a alembic directory within the todoApp
- It is to be done by writing a command : alembic init <name>.

```
(env) ericroby@Erics-MacBook-Pro TodoApp % alembic init alembic
```

- Alembic revision :
- Alembic revision is how we create a new alembic file where we can add some type of database upgrade

```
ericroby@FastAPI-Project ~%
alembic revision -m "create phone number col on users table"
```

- When we will execute to create a new file.
- This new file where we can write the upgrade code.
- This file is going to have a revision id attached to it.
- Alembic upgrade function:
- This upgrade function is used for upgrading the database
- Alembic upgrade is how we actually run the migration

```
def upgrade() -> None:
  op.add_column('users', sa.Column('phone_number', sa.String(), nullable=True))
```

- Sa stands for sqlalchemy.
- we're making nullable as equal to true since immediately after creation there's not going to any data for the phone number for each user that's already created.
- To run the migration we will give the following command

```
ericroby@FastAPI-Project ~%
alembic upgrade <revision id>
```

- This command will successfully implement the changes within the upgrade functionality.
- Alembic downgrade function:
- Alembic downgrade is how we revert a migration.
- Revert our database o remove the last migration change.
- We will write the downgrade function within the same file.

```
def downgrade() -> None:
  op.drop_column('users', 'phone_number')
```

- To run downgrade we will use the given command.

```
ericroby@FastAPI-Project ~%
alembic downgrade -1
```

PROJECT 4-UNIT AND INTEGRATION TESTING

- TESTING:
- FastAPI testing includes unit testing and integration testing.
- Testing is a way to make sure that the application is running or working as intended.
- Testing is a part of Software Development Lifecycle (SDLC) that aims to identify :

1. Errors
2. Bugs
3. Defects

- Make sure that application meets user requirements and specifications
- Ensuring software is of high quality , reliable, secure and user-friendly.
- Manual testing is the testing we do manually ourself.

- Unit Testing:
- Unit testing involves testing of individual components and units of software in isolation from the rest of the application.
- Each unit is the smallest testable part of the software such as functions and methods.
- Validates that each unit of the software performs as designed.
- Developers write unit test in development phase.
- Tests are automated and are executed by a testing framework that is **pytest.**
- Benefits of unit testing is identifying bugs early in development phase.

- Integration Testing:
- Focuses on testing the interaction and interface between different units and components of the software
- Integration testing combines multiple units together and test them as a group.
- By testing integrated components it helps uncover the issues that may arise from their interactions and this contributes to the overall stability and correctness of the solution.
- Example: Calling an api endpoint to ensure correct solution, as it involves testing the interaction between the client and the server.

- Pytest:
- Pytest is a popular python testing framework .
- It is known for its simplicity ,scalability and ability to handle both unit and integration testing.
- It offers features likenative assertion for simple and flexible testing and fixtures for efficient test setup and teardown.

- PYTEST BASICS:
- Unit testing is how we test the application even without running the application
- For doing this we will run a script that is testing script
- For doing this the first thing is to create a directory and name it as test directory
- In this directory we will create a python file that is init.py
- And we will create a python file and we will name it as test_<name> ,since it is an testing file we will name it as test_ some name .
- Then the next step is to install pytest by using "**pip install pytest**".
- For checking if the value is equal or not:

```
def test_equal_or_not_equal():
    assert 3 == 3
```

- And in terminal just write pytest .
- We can validate the following:
- Validate integers:

```
def test_equal_or_not_equal():
    assert 3 == 3
```

Pass

- Validate instances:

```
def test_is_instance():
    assert isInstance('this is a string', str)
    assert not isInstance('10', int)
```

- Validate booleans:

```
def test_boolean():
    validated = True
    assert validated is True
    assert ('hello' == 'world') is False
```

- Validates types:

```
def test_type():
    assert type('Hello' is str)
    assert type('World' is not int)
```

- Assert: assert is used in pytest to verify expectations and validate conditions within your tests. It serves as the primary purpose mechanism to determine whether test fails or pass.

- Validate greater and less than:

```
def test_greater_and_less_than()
    assert 7 > 3
    assert 4 < 10
```

- Validate types:

```
def test_list():
    num_list = [1, 2, 3, 4, 5]
    any_list = [False, False]
    assert 1 in num_list
    assert 7 not in num_list
    assert all(num_list)
    assert not any(any_list)
```

- Pytest objects:
- Create our python object.

```
import pytest

class Student:
    def __init__(self, first_name: str, last_name: str, major: str, years: int):
        self.first_name = first_name
        self.last_name = last_name
        self.major = major
        self.years = years
```

```
def test_person_initialization():
    p = Student('John', 'Doe', 'Computer Science', 3)
    assert p.first_name == 'John', 'First name should be John'
    assert p.last_name == 'Doe', 'Last name should be Doe'
    assert p.major == 'Computer Science'
    assert p.years == 3
```

- For this instead of initializing a instance you can create a class with a constructor.
- Then we can also pytest fixture ,fixture is used for avoiding the use of p
-

```
@pytest.fixture
def default_employee():
    return Student('John', 'Doe', 'Computer Science', 3)
```

```
def test_person_initialization(default_employee):
    assert default_employee.first_name == 'John', 'First name should be John'
    assert default_employee.last_name == 'Doe', 'Last name should be Doe'
    assert default_employee.major == 'Computer Science'
    assert default_employee.years == 3
```

- First test file is checking a connection from main.py file to check the connection healthy .
- For this ,first we will go in main.py and we will create a new function .

```python
@app.get("/healthy")
def health_check():
    return {'status': 'Healthy'}
```

- The in our test directory we will create a new file as test_main.py.
- In this we will create client for testing . but this will give us error ,to solve the issue you will have refactor the directory and set the path correctly.

```python
from fastapi.testclient import TestClient
import main
from fastapi import status

client = TestClient(main.app)

def test_return_health_check():
    response = client.get("/healthy")
    assert response.status_code == status.HTTP_200_OK
    assert response.json() == {'status': 'Healthy'}
```

- For testing the application we will create a fake directory that can store data.
- Creating testing dependency that are different from that of production database.
- This way integration can make sure the entire the program is working perfectly.
- That means we will have two databases :one for app production and is only sed when app is live, and the other one for testing and is only when app is testing.
- For this we will create a new file in test directory and name it as test_todos.py as we will use it for testing todos.py file.
- For this we will create a sqlite database in the file:

```python
SQLALCHEMY_DATABASE_URL = "sqlite:///./testdb.db"
```

- We will create it in the same way as we created the production database.

```
from sqlalchemy import create_engine
from sqlalchemy.pool import StaticPool
from ..database import Base

SQLALCHEMY_DATABASE_URL = "sqlite:///./testdb.db"
engine = create_engine(
    SQLALCEMY_DATABASE_URL,
    connect_args={"check_same_thread": False},
    Poolclass = StaticPool,
)
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

- For testing the api endpoints of todos file we will have to create and override get_db and get_current_user.and we will use this for making changes in the testing database.

```
def override_get_db():
    db = TestingSessionLocal()
    try:
        yield db
    finally:
        db.close()

def override_get_current_user():
    return {'username': 'codingwithroby', 'id': 1, 'user_role': 'admin'}

app.dependency_overrides[get_db] = override_get_db

app.dependency_overrides[get_current_user] = override_get_current_user
```

- For normal implementation we use app but for testing we will use client as testclient

```
app = FastAPI()
```

```
client = TestClient(app)
```

- Git :
- Free and open-source distributed version control system
- Can handle small and large projects
- Allow team members to use same files by distributed braches/environmemnts
- In git we can commit every time a new file is added or the file is update and we can do it without worrying about breakage.
- Git is a important technology for software engineer.
- Git basics:
- Every time you commit the code that is called a version and every time you commit the updated change version numbe changes.
- r

Git Command	Details
git init	Initializes a new, empty repository
git add .	Adds files from a non-staged area to a staging GIT area
git commit –m “info here”	Move files from a staging area to a commit
git checkout <commit #>	Open up previous commit
git log	Shows committed Snapshots

- Render
- Render is a PaaS that is platform as a service.
- A platform as a service is a main category of cloud computing
- Help developers build ,run and operate the application entirely on cloud
- This means instead of hosting application from your pc it application deployment is done by cloud.
- This allows developers to focus on coding and not to worry about the infrastructure of the application.
- Many companies relies on cloud computing like render to handle the entire infrastructure of the application
- The pricing depends on many factors of how your application o perform online
- We will use free trial with free plan
- Businesses of all sizes use render as their cloud PaaS provider.
- We will have a requirement file that will consist of the all the deployment of the application.

```
o deployment % pip3 freeze > requirements.txt
```